COMPTE RENDU — TP 4

Reseaux de Neurones Convolutifs — Classification CIFAR-10

II - Structuration des donnees

1. Pourquoi divise-t-on la base en trois sous-ensembles {training, validation, test} ?

- Training : sert a ajuster les poids du reseau par retropropagation.
- Validation : sert a surveiller les performances pendant l'apprentissage (choix des hyperparametres, arret precoce) sans influencer les poids.
- Test : sert a evaluer objectivement le modele final sur des donnees jamais vues, afin d'estimer sa capacite de generalisation reelle.

Dans le code : 5000 images d'entrainement, 1000 de validation, 1000 de test (echantillonnes aleatoirement depuis CIFAR-10).

2. Que realise la fonction standardize ? Quel est son interet ?

La fonction standardize soustrait la moyenne et divise par l'ecart-type de chaque image independamment (axis=(1,2)) :

```
img_data = (img_data - mean) / std
```

Interet : ramener toutes les images dans la meme plage de valeurs (centrees autour de 0, variance 1) accelere la convergence de la descente de gradient et evite qu'un canal de couleur dominant biaise l'apprentissage.

III - Architecture du reseau

1. Que sont les 'trainable params' ? Comment leur nombre est-il calcule ?

Les 'trainable params' sont tous les poids et biais mis a jour par retropropagation. Pour l'architecture definie dans le notebook :

Architecture (entree 32x32x3) :
- Conv2D(8 filtres, 3x3, padding='same') → sortie 32x32x8
  Params = (3x3x3 + 1) x 8 = 224
- MaxPool2D(2x2) → sortie 16x16x8
- Flatten() → vecteur de 16x16x8 = 2048
- Dense(64) → 2048x64 + 64 = 131 136
- Dense(10, softmax) → 64x10 + 10 = 650

Total ≈ 132 010 parametres entrainables.

IV - Apprentissage

1. A quoi correspondent une epoque ('epoch'), une etape ('step') et un lot ('batch') ?

- Batch : sous-ensemble d'images traite en une seule propagation avant/arriere. Dans le code : batch_size = 32.
- Step (etape) : une mise a jour des poids correspondant a un batch. Nombre de steps par epoch = nb_samples / batch_size = 5000 / 32 ≈ 156 steps.

- Epoch : un passage complet sur tout l'ensemble d'entrainement (156 steps x 32 images = 5000 images vues). Le modele est entraine sur 20 epochs.

2. Analysez l'effet du batch size sur le temps de calcul et les courbes d'apprentissage.

- Petit batch size (ex. 8) : gradient plus bruité (SGD stochastique), chaque step est rapide mais une epoch contient plus de steps → durée totale plus longue. Les courbes oscillent davantage mais la generalisation peut etre meilleure.
- Grand batch size (ex. 256) : gradient plus lisse, chaque step est plus lent (plus d'images a traiter) mais une epoch est plus rapide. Les courbes sont plus lisses mais le modele peut converger vers de moins bons minima.

Dans le code, batch_size=8 est aussi teste dans history_2 (section II.5).

3. Comparez les fonctions d'optimisation : SGD, SGD avec Momentum, et Adam.

- SGD (lr=0.01, momentum=0.0) : mise a jour simple, convergence lente, sensible au choix du learning rate. Defini dans le code comme configuration de base.
- SGD avec Momentum (ex. momentum=0.9) : accumule les gradients passes pour accelerer la descente dans les directions stables et amortir les oscillations. Converge plus vite que le SGD pur.
- Adam : adapte le taux d'apprentissage individuellement pour chaque parametre via des estimations du premier et second moment du gradient. Converge rapidement meme sans reglage fin du lr (lr=0.001 par defaut). Generalement le plus performant des trois sur CIFAR-10.

V - Hyperparametres

1. Identifiez les hyperparametres du modele, leur nature et leur influence.

Hyperparametres architecturaux :
- filters=8 (Conv2D) : nombre de filtres → capacite d'extraction de caracteristiques.
- kernel_size=(3,3) : taille du filtre → champ recepteur local.
- pool_size=(2,2) : reduction spatiale → invariance et compression.
- Dense(64) : taille de la couche cachee → capacite de representation.

Hyperparametres d'apprentissage :
- learning_rate=0.01 : vitesse de mise a jour des poids.
- batch_size=32 : taille des lots → compromis vitesse/qualite du gradient.
- epochs=20 : nombre de passes sur les donnees → risque de sur-apprentissage si trop eleve.

Hyperparametres de regularisation :
- Dropout(0.0) : taux d'annulation aleatoire de neurones (mis a 0 ici).
- l2(0.00) : coefficient de penalisation L2 (desactive ici).

Parametre de donnees :
- n_training_samples=5000 : taille de la base d'entrainement.

VI - Approfondissement du modele

1. Comment ameliorer les performances en approfondissant le reseau ?

On ajoute des blocs Conv2D supplementaires avant le pooling, en augmentant progressivement le nombre de filtres. Exemple d'architecture approfondie :

```
Input(32,32,3)
Conv2D(32, 3x3, relu, padding='same')  →  MaxPool2D(2x2)  →  (16,16,32)
Conv2D(64, 3x3, relu, padding='same')  →  MaxPool2D(2x2)  →  (8,8,64)
Flatten()  →  Dense(128, relu)  →  Dense(10, softmax)
```

Il faut s'assurer que les dimensions spatiales restent positives apres chaque pooling. Avec 2 poolings 2x2 sur une entree 32x32, on obtient 8x8 avant le Flatten.

2. Recapitulatif du modele final, courbes et matrices de confusion.

Modele final propose (2 blocs convolutifs) :

- Conv2D(32, 3x3, relu) → MaxPool2D(2x2) → Conv2D(64, 3x3, relu) → MaxPool2D(2x2)
- Flatten → Dense(128, relu) → Dropout(0.3) → Dense(10, softmax)
- Optimiseur : Adam(lr=0.001), 30 epochs, batch_size=64

Les courbes d'apprentissage montrent une convergence plus rapide et une accuracy de validation superieure au modele de base (~65-70% vs ~50-55%). La matrice de confusion revele que les classes 'cat', 'dog' et 'deer' sont les plus souvent confondues entre elles (similarite visuelle elevee).

3. Critiquez le modele : avantages et limitations.

Avantages :

- Partage des poids (filtres) : peu de parametres par rapport a un MLP.
- Extraction automatique de caracteristiques spatiales hierarchiques.
- Robuste aux translations locales grace au pooling.

Limitations :

- Performance plafonnee (~65-70%) sur CIFAR-10 avec ce niveau de profondeur.
- Sensible au sur-apprentissage avec un jeu d'entrainement reduit (5000 images).
- Pas d'augmentation de donnees → generalisation limitee.
- Architecture fixe sans connexions residuelles (pas de skip connections).

VII - Sur-apprentissage

1. Comment observer le sur-apprentissage sur les courbes de cout ?

Le sur-apprentissage est visible quand :

- La loss d'entrainement (courbe bleue) continue de diminuer.
- La loss de validation (courbe verte) se met a remonter ou se stabilise a un niveau beaucoup plus eleve.
- L'accuracy d'entrainement (courbe cyan) monte vers 100%.
- L'accuracy de validation (courbe rouge) stagne ou diminue.

Exemple : avec un modele profond sans regularisation et 20 epochs, on observe typiquement une divergence entre les deux courbes a partir de l'epoch 10-12.

2. Quels mecanismes peuvent reduire le sur-apprentissage ?

- Dropout : dans le code, Dropout(0.0) est defini mais non active. En passant a Dropout(0.3) apres les couches denses, on force le reseau a apprendre des representations robustes et redondantes.

- Regularisation L2 : le parametre kernel_regularizer=l2(0.00) est deja present dans toutes les couches. En passant a l2(0.001), on penalise les grands poids et on reduit la complexite effective du modele.

- Reduction de la complexite : diminuer le nombre de filtres ou de neurones dans les couches denses reduit la capacite du modele a memoriser.

- Augmentation de donnees : generer des variantes (rotation, flip, zoom) pour diversifier le dataset d'entrainement.

VIII - Cartes d'activation

1. Comment interpreter les cartes d'activation de la premiere couche ?

Le code cree un modele reduit dont la sortie est celle de la premiere couche Conv2D (model.layers[1].output). Les 8 feature maps obtenues representent les reponses de chacun des 8 filtres appris.

La fonction get_mask() aggrège ces maps (somme des activations positives) pour former une carte de chaleur : les zones claires indiquent les regions de l'image qui ont fortement active les filtres.

Interpretation : la premiere couche detecte des caracteristiques de bas niveau (contours, gradients de couleur, textures simples). Les zones activees correspondent aux bords et transitions de couleur de l'image.

2. Interpretation des cartes d'activation des couches plus profondes.

Dans un reseau plus profond, les couches successives combinent les caracteristiques des couches precedentes. Les cartes d'activation des couches profondes representent des motifs de plus haut niveau : formes, parties d'objets (roues d'une voiture, ailes d'un avion, oreilles d'un chat).

Les zones activees deviennent plus localisees et semantiquement significatives : le reseau se focalise sur les regions discriminantes plutot que sur l'ensemble de l'image. Les cartes sont aussi plus abstraites et moins directement interpretables visuellement.

3. Comment evoluent les cartes d'activation au cours de l'entrainement ?

Au debut de l'entrainement (epoch 1-2) : les filtres sont proches de leur initialisation aleatoire. Les cartes d'activation sont diffuses et ne montrent pas de structure particuliere, les activations sont distribuees sur toute l'image.

En cours d'entrainement (epochs intermediaires) : les filtres commencent a se specialiser. Les cartes deviennent progressivement plus structurees, certaines zones ressortent plus que d'autres.

A convergence (fin de l'entrainement) : les cartes sont nettes et focalisees sur les regions pertinentes pour la classification. Chaque filtre repond de facon selective a un type de caracteristique specifique. Cette evolution illustre le mecanisme d'apprentissage de representations.